

Design and Implementation of a Parallel Prefix Adder

Gauri Krishnan

2024102073

IIT Hyderabad

gauri.krishnan@students.iit.ac.in

Abstract—This project focuses on the design and implementation of a 5-bit Carry Look-Ahead Adder (CLA), a high performance digital circuit for binary addition. The CLA architecture is chosen for its ability to significantly reduce propagation delay compared to traditional ripple carry adders, making it suitable for high-speed arithmetic operations in modern digital systems. Performance metrics, such as delay, power consumption, gate count, are evaluated to demonstrate the efficiency of the CLA. Data collection and simulations have been done using tools like NGSPICE, MAGIC layout tool and AMD Xilinx Vivado. This project aims to provide insights into efficient adder design techniques.

I. INTRODUCTION

The design of high-performance adders is a central problem in digital arithmetic, where reducing carry-propagation delay directly impacts overall system speed. Parallel prefix adders, such as the Brent–Kung architecture, address this challenge by organizing carry computation as a structured prefix network that operates in logarithmic time. Drawing on the foundational ideas in Brent and Kung’s VLSI-oriented model, this adder employs a balanced tree structure that minimizes fanout, reduces wiring complexity, and enables a highly regular physical layout. The Brent–Kung topology achieves an attractive trade-off: it computes carries in $O(\log n)$ stages while using significantly fewer wiring resources and lower area than other parallel prefix structures. These characteristics make it well suited for VLSI implementation and high-speed digital systems where regularity, reduced fanout, and controlled area are critical.

II. FLIP-FLOP DESIGN AND USAGE

A. Integration of Flip-Flops for Timing Stability

Flip-flops are incorporated at both the input and output stages of the Adder to ensure reliable high-speed operation and to eliminate timing-related vulnerabilities.

Registering the input signals A_i, B_i, C_i aligns all incoming data to a common system clock. This synchronization mitigates the risk of metastability, removes input-arrival skew, and guarantees that the CLA logic processes a fully coherent input vector at each clock cycle.

The output flip flops capture the generated sum bits (S_0, S_1, S_2, S_3, S_4) and the carry out, C_5 at the positive clock edge. This mechanism ensures that the outputs remain stable throughout the clock period, preventing transient glitches from affecting downstream circuitry.

By clocking both boundaries, the design effectively encapsulates the combinational Adder logic between two synchronous register stages. This structure eliminates race conditions, enforces deterministic timing behavior, and enhances the robustness of the overall arithmetic unit, particularly under high-frequency operating conditions.

B. Working of a TSPC D-Flip-Flop

For this project, a True Single-Phase Clock (TSPC) D-Flip Flop is utilized to provide both input and output synchronization for the Parallel-Prefix Adder. The selection of the TSPC architecture is motivated by its efficiency and suitability for high-performance designs. Its single-clock operation minimizes control complexity, while its compact structure reduces silicon area and power consumption. Furthermore, the TSPC flip-flop supports rapid switching, enabling reliable operation at elevated clock frequencies, which is essential for modern high-speed digital arithmetic circuits.

The implementation chosen is a positive edge triggered flip flop since the input is given before the positive edge of the clock and the output should be evaluated by the next clock edge.

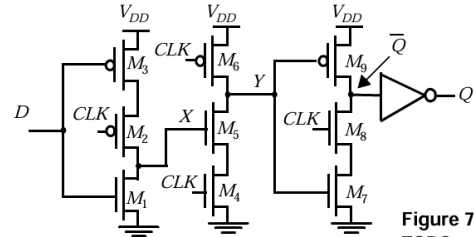


Fig. 1. TSPC D Flip Flop

- When the clock is low ($CLK = 0$), the left latch formed by transistors M_1, M_2 , and M_3 becomes active. In this phase, M_2 conducts and the input D is sampled onto node X . M_6 is ON, so Y gets pre-charged to V_{DD} .
- When the clock transitions high ($CLK = 1$), transistor M_2 turns off. At the same time, the middle stage (M_4-M_6) is enabled, allowing the value stored at node X to propagate to node Y . The output stage (M_7-M_9) is also activated during this phase, driving the stored value to the output. Any changes at the input D during the high clock phase

do not influence the output, because the sampling path is fully disabled.

C. Timing Constraints

The TSPC D-Flip-Flop imposes specific timing requirements that must be met to ensure correct and reliable operation:

- Setup Time (t_{su}): The minimum time the input data (D) must be stable before the clock edge.
- Hold Time (t_h): The minimum time the input data (D) must remain stable after the clock edge.
- Clock-to-Q Propagation Delay (t_{PCQ}): The time it takes for the output (Q) to reflect a change in the input (D) after the clock edge.

D. Sizing the Flip-Flop

Initially, the ratio chosen was with reference to a minimum sized inverter, i.e, the widths of the PMOS transistors are scaled to twice the widths of the NMOS transistors in order to match the current carrying capabilities. This minimizes the difference in the rise and fall delays.

Stage	PMOS	NMOS
1	40λ	10λ
2 and 3	20λ	20λ

TABLE I
INITIAL RATIOS

However, glitches may occur at the output due to a race condition when the clock transitions from low to high [4]. Consider the case where D is low and $Q = 1$ ($Q=0$). While CLK is low, Y is pre-charged high turning on M7. When CLK transitions from low to high, nodes Y and $\bar{Q}$ start to discharge simultaneously (through M4-M5 and M7-M8, respectively). Once Y is sufficiently low, $\bar{Q}$ is pulled high through M9.

Fig 2 shows how $\bar{Q}$ dips for a short period of time. This glitch could create unwanted events and could cause problems in our circuit. This can be reduced by making the pull-down transistors in stage 3 weaker than those in stage 2 so that Y discharges much faster than $\bar{Q}$.

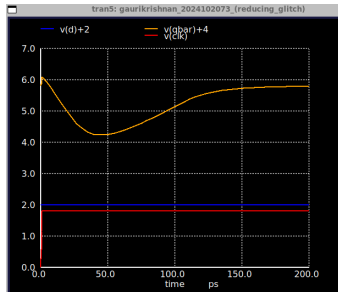


Fig. 2. Initial Glitch

Final ratio after taking area also into consideration:

As can be observed from , the dip in $\bar{Q}$ is much lesser after changing the ratios.

The following tables show t_{su} , t_h , t_{PCQ} for TSPC DFF pre and post layout.

Stage	PMOS	NMOS
1	40λ	10λ
2	20λ	20λ
3	20λ	10λ

TABLE II
INITIAL RATIOS

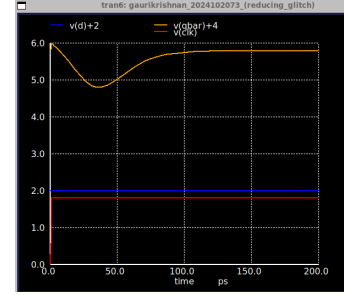


Fig. 3. Reduced Glitch

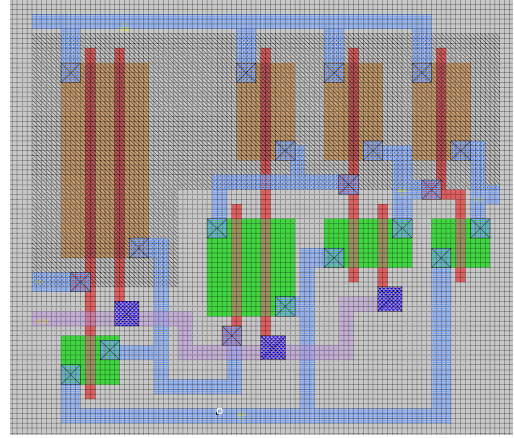


Fig. 4. MAGIC Layout of TSPC DFF

t_{su}	53ps
t_h	37ps
t_{PCQ}	147ps

TABLE III
PRE-LAYOUT VALUES

t_{su}	50.9ps
t_h	28.1ps
t_{PCQ}	100ps

TABLE IV
POST-LAYOUT VALUES

III. WHY PARALLEL-PREFIX ADDER?

Carry Computation Time of a Parallel Prefix Adder

Consider two n -bit operands with bitwise generate and propagate signals

$$g_i = a_i b_i, \quad p_i = a_i \oplus b_i, \quad 0 \leq i \leq n-1.$$

The full-adder carry recurrence is

$$c_{i+1} = g_i \vee (p_i c_i), \quad c_0 = C_{in}.$$

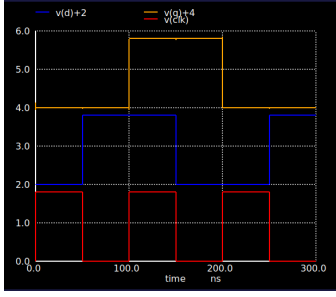


Fig. 5. Pre-Layout

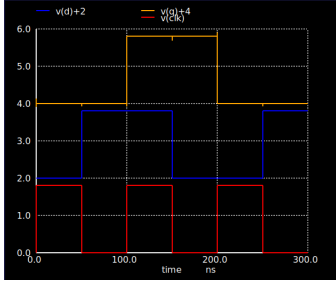


Fig. 6. Post-Layout

To remove the sequential dependence of this recurrence, we define a binary operator on generate-propagate pairs:

$$(g, p) \circ (g', p') = (g \vee (pg'), pp').$$

This operator is *associative*, so groups of bits may be combined in any tree-shaped parenthesization.

1. *Prefix pairs*: Define the *prefix pair* for bits 0 through i as

$$(G_i, P_i) = (g_i, p_i) \circ (g_{i-1}, p_{i-1}) \circ \cdots \circ (g_0, p_0).$$

A fundamental identity of the prefix method shows that the carry into bit $i + 1$ is:

$$c_{i+1} = G_i \vee (P_i c_0).$$

Thus: - If $c_0 = 0$, then $c_{i+1} = G_i$. - If $c_0 \neq 0$, the carry is adjusted by the term $P_i c_0$.

In both cases, once (G_i, P_i) are known, all carries follow immediately.

2. *Depth of the prefix computation*: $O(\log n)$: At the leaves of the network lie the pairs (g_i, p_i) . At level 1 of the prefix tree, adjacent pairs are combined to form prefixes of length 2:

$$(g_i, p_i) \circ (g_{i-1}, p_{i-1}).$$

At level 2, prefixes of length 4 are formed by combining two length-2 prefixes. More generally, level k produces prefixes spanning 2^k bits.

After k levels, the largest prefix that can be constructed covers 2^k bits. To cover all n bits, we require

$$2^k \geq n \implies k \geq \log_2 n.$$

Thus, the prefix computation of all (G_i, P_i) requires

$$\lceil \log_2 n \rceil \text{ levels}.$$

Each level contains only constant-time Boolean operations (AND/OR), so the prefix computation time is

$$T_{\text{prefix}}(n) = O(\log n).$$

3. *Effect of an arbitrary carry-in c_0* : Once (G_i, P_i) are available, each carry is computed using

$$c_{i+1} = G_i \vee (P_i c_0),$$

which requires only one AND and one OR per bit. This post-processing adds only *constant* depth, independent of n .

Thus the total carry computation time is

$$T(n) = O(\log n) + O(1) = O(\log n).$$

Therefore, a parallel prefix (Brent-Kung) adder computes all carries in $O(\log n)$.

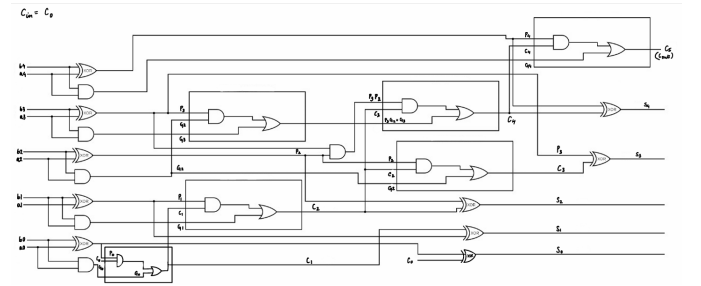


Fig. 7. Circuit diagram of Adder

IV. GATE IMPLEMENTATION

The gates needed for this circuit are AND, OR, and XOR.

A. XOR:

A GDI-based XOR structure [2] is employed in this design, followed by two cascaded inverters to restore full logic levels. Advantages:

- GDI uses only 4 transistors to realise XOR function whereas Static CMOS uses 12 - saves on area.
- Lower dynamic power consumption and improved switching speed

Disadvantages:

- Unlike static CMOS logic, GDI gates apply one of the input signals to a source or drain terminal rather than to an insulated gate. As a result, the driven node is no longer isolated, leading to a finite input impedance and possible current flow through the conducting channel. This can introduce voltage degradation and reduced noise margins, often requiring additional buffering or level restoration to ensure reliable cascading of GDI stages.
- Since we do not obtain rail-to-rail voltages and thus, we use a buffer after the XOR logic. This brings the total count of transistors to 8, which is still lesser than the no. of transistors in a Static CMOS implementation.

B. AND, OR:

Static CMOS logic is used for implementing the AND and OR functions.

Advantages:

- Static CMOS implementation ensures rail-to-rail voltage outputs.
- Low static power consumption as current flows only during switching events.
- High noise tolerance compared to other logic styles.

Disadvantages:

- Relatively larger transistor count and increased area compared to alternative logic styles.
- Increased layout complexity associated with static CMOS can impact overall design density in large-scale circuits.

The following figures show the stick diagrams and MAGIC Layouts of the gates used:

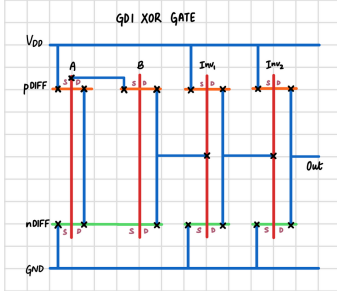


Fig. 8. XOR stick diagram

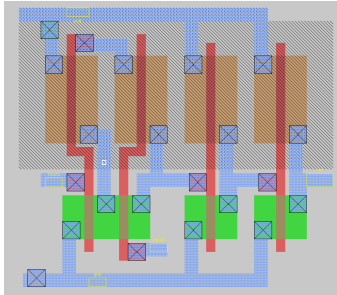


Fig. 9. XOR MAGIC Layout

The ratio of all the gates was taken with reference to a minimum size inverter with PMOS sized with twice the width of an NMOS transistor to equalise rise and fall time delays.

Ideally, the transistor dimensions in successive stages of a circuit should be progressively increased to ensure adequate drive strength and to accommodate the larger capacitive loads encountered at deeper levels of logic. This sizing strategy helps maintain signal integrity and minimizes delay. However, for simplicity and uniformity in this design, a fixed transistor width of $W_n = 10\lambda$ was adopted across all stages.

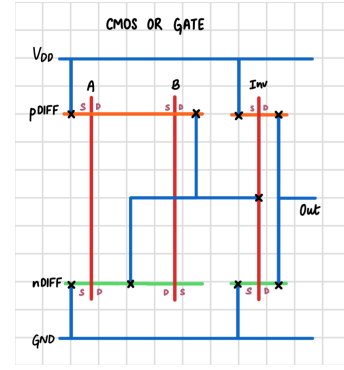


Fig. 10. OR stick diagram

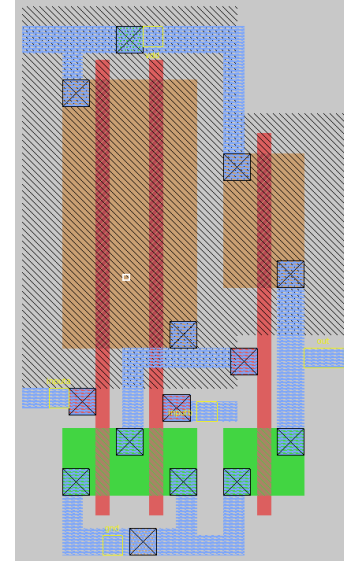


Fig. 11. OR MAGIC Layout

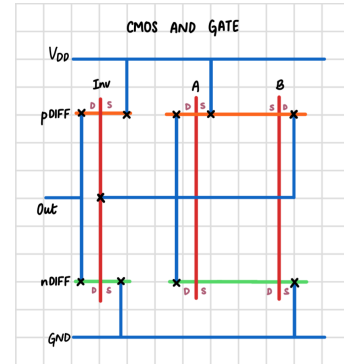


Fig. 12. AND stick diagram

V. DELAY CHARACTERIZATION AND OPTIMAL CLOCK SPEED

As discussed earlier, the TSPC D-Flip-Flop imposes specific timing requirements that must be met to ensure correct and reliable operation. We must therefore, ensure that hold time and set-up time are not violated.

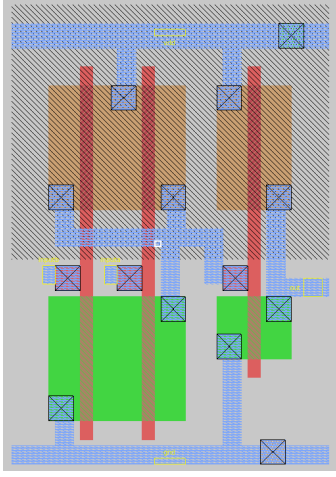


Fig. 13. AND MAGIC Layout

$$T_{clk} \geq t_{su} + t_{PCQ} + t_{pd_{max}}$$

and

$$t_h \leq t_{PCQ} + t_{pd_{min}}$$

The worst case delay of the adder would be along the path from the input A0 to C5 (C_{out}), and the min. delay would be along the path A0 to the sum S0.

The values measured pre-layout are:

$t_{pd_{max}}$	1003p
$t_{pd_{min}}$	363p

TABLE V

PRE-LAYOUT MIN AND MAX ADDER PROPAGATION DELAY

A quick observation tells us that there is no hold time violation since the inequality for t_h holds true.

Now, for the maximum clock speed(the fastest it can be for full functionality of the circuit), $T_{clk} \geq 53p + 147p + 1003p$

Therefore, the minimum clock time period, $T_{clk_{min}} = 1.203ns$.

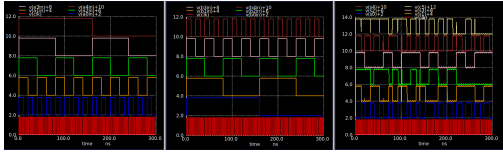


Fig. 14. Pre-Layout ngspice to verify adder functionality

VI. POST-LAYOUT

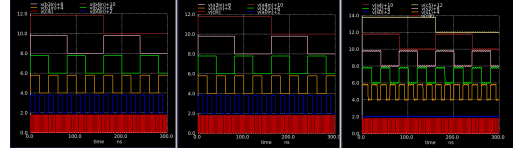


Fig. 15. Post-Layout ngspice to verify adder functionality

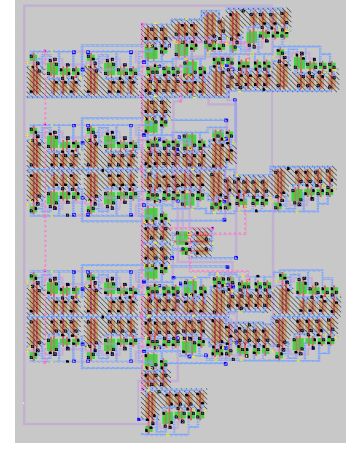


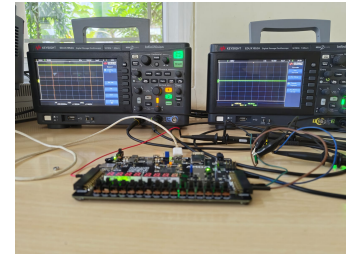
Fig. 16. Full Magic Layout

TABLE VI
PLACEHOLDER CAPTION

Parameter	Pre-Layout	Post-Layout
Minimum propagation delay	363ps	450ps
Maximum propagation delay	1.003ns	1.450 ns
Flip-flop setup time	53ps	51ps
Flip-flop hold time	37ps	28ps
Clock-to-Q delay	147ps	100ps
Minimum clock period ($T_{clk-min}$)	1.203ns	1.601ns
Max Frequency	0.831GHz	0.731GHz

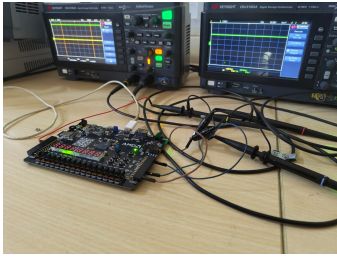
VII. IMPLEMENTING THE CIRCUIT ON FPGA

The given circuit has been implemented in the FPGA successfully.



REFERENCES

- [1] Brent and Kung, "A Regular Layout for Parallel Adders," in IEEE Transactions on Computers, vol. C-31, no. 3, pp. 260-264, March 1982, doi: 10.1109/TC.1982.1675982.
- [2] A. K. Nishad and R. Chandel, "Analysis of Low Power High Performance XOR Gate Using GDI Technique," 2011 International Conference on Computational Intelligence and Communication Networks, Gwalior, India, 2011, pp. 187-191, doi: 10.1109/CICN.2011.37.



- [3] P. M. Kogge and H. S. Stone, "A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations," in *IEEE Transactions on Computers*, vol. C-22, no. 8, pp. 786-793, Aug. 1973, doi: 10.1109/TC.1973.5009159.
- [4] Jan M. Rabaey, Anantha Chandrakasan, and Borivoje Nikolić. *Digital Integrated Circuits: A Design Perspective*. 3rd edition. Pearson, 2016.
- [5] Mano, M. Morris, and Michael D. Ciletti. *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*. 5th edition. Pearson, 2012.
- [6] B. Razavi, "TSPC Logic [A Circuit for All Seasons]", in *IEEE Solid State Circuits Magazine*, vol. 8, no. 4, pp. 10-13, Fall 2016.
- [7] Ivan E. Sutherland, Bob F. Sproull and David L. Harris. *Logical Effort: Designing Fast CMOS Circuits*, Morgan Kaufmann Publishers, 1998.